Applied Cryptography
Lect. #6

Marco Calderini

University of Trento

A.Y. 2024/2025

Last time:

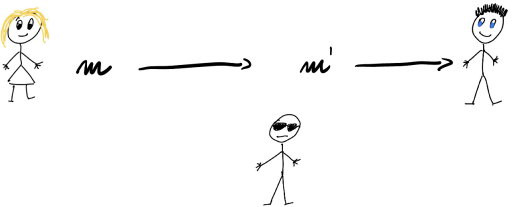
- Block ciphers,
- CPA security

This time:

- Message authentication codes (MAC)

Message Integrity

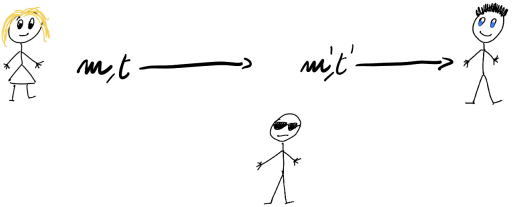
Message Integrity



Goal: integrity, no confidentiality.

Independent of whether messages remain confidential, we want to ensure that they can't be modified by a third party in transit.

Message Integrity



Solution: Add a special **tag** to ensure integrity of the message.

Cryptographic and non-cryptographic integrity checking

Non-cryptographic protocols also often include integrity checks:

- Ethernet uses CRC32, a 32-bit checksum
- TCP has a 16-bit checksum

These algorithms protect against *random* errors, but they are public and do not include keys so can be forged by a malicious adversary.

Cryptographic and non-cryptographic integrity checking

In order to protect against malicious forgery, cryptographic integrity checks <u>must</u> use a secret key.

Encryption does not suffice for integrity checking.

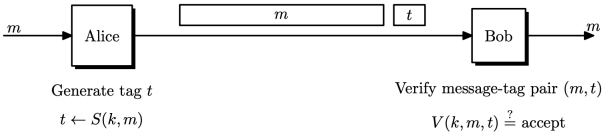
Example (Stream cipher encryption)

$$Enc_k(m) = G(k) \oplus m = c$$

$$Dec_k(c \oplus anything) = m \oplus anything$$

Message authentication codes

Message integrity: MACs



Definition

A MAC system $\mathcal{I} = (S, V)$ defined over $(\mathcal{K}, \mathcal{M}, \mathcal{T})$ is a pair of efficient algorithms, S and V , where

- S (probabilistic) signing algorithm: generates tags, $t \xleftarrow{R} S(k, m)$
- V (deterministic) verification algorithm: verifies tags, $V(k, m, t) = r$ where $r \in \{\text{accept, reject}\}$

Remark: S can be deterministic. In this case, there is a unique tag t for any message m.

Secure MACs

Attacker's power: **chosen message attack**

- for $m_1, m_2, ..., m_Q$ attacker is given ti $t_i \xleftarrow{R} S(k, m_i)$

Attacker's goal: **existential forgery**

- produce some new valid message/tag pair (m, t), with

$$(m, t) \notin \{(m_1, t_1), ..., (m_Q, t_Q)\}$$

$\Rightarrow$ attacker cannot produce a valid tag for a new message

$\Rightarrow$ given (m, t) attacker cannot even produce (m, t') for $t' \neq t$

Secure MACs

Attack Game (MAC security)

Given a MAC system $\mathcal{I} = (S, V)$, defined over $(\mathcal{K}, \mathcal{M}, \mathcal{T})$, and an adversary $\mathcal{A}$, the attack game runs as follows:

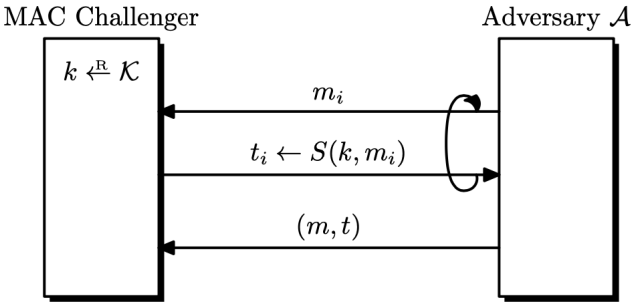
- The challenger picks a random $k \xleftarrow{R} \mathcal{K}$.
- $\mathcal{A}$ queries the challenger several times.
  - For $i = 1, 2, ..., Q$ the ith signing query is a message $m_i \in \mathcal{M}$.
  - Given m_i, the challenger computes a tag $t_i \xleftarrow{R} S(k, m_i)$, and then gives t_i to $\mathcal{A}$.
- Eventually $\mathcal{A}$ outputs a candidate forgery pair $(m, t) \notin \{(m_1, t_1), ..., (m_Q, t_Q)\}$

$\mathcal{A}$ wins the game if $V(k, m, t) =$ accept ((m, t) valid pair under k).
$MACadv[\mathcal{A}, \mathcal{I}]$ denotes the probability that $\mathcal{A}$ wins the game.

Definition

The MAC system $\mathcal{I}$ is secure if for all efficient adversaries A, the value $MACadv[\mathcal{A}, \mathcal{I}]$ is negligible.

Secure MACs


MAC Challenger
Adversary $\mathcal{A}$
$k \xleftarrow{R} \mathcal{K}$
m_i
$t_i \leftarrow S(k, m_i)$
(m, t)

Remark

- *pair* (m, t) *is called an existential forgery*
- *Secure MAC is said existentially unforgeable under a chosen message attack.*
- *for deterministic MACs we require* $m \notin \{m_1, ..., m_Q\}$
- *MAC verification queries do not help the attacker*
  Attacker can send pairs $(\hat{m}, \hat{t})$ *to the challenger and receives* $V(k, \hat{m}, \hat{t})$ *and wins if* $V(k, \hat{m}, \hat{t}) = accept$.

$$MACadv^{vq}[\mathcal{A}, \mathcal{I}] \leq MACadv[\mathcal{B}, \mathcal{I}] \cdot Q_v.$$

MACs based on PRFs

Secure MAC from secure PRF

Let F be PRF defined over $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$. We define the *deterministic* MAC system $\mathcal{I}_F = (S, V)$ as:

- $S(k, m) = F(k, m)$;
- $$V(k, m, t) = \begin{cases} \text{accept} & \text{if } F(k, m) = t, \\ \text{reject} & \text{otherwise,} \end{cases}$$

A bad example

Suppose $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ is a secure PRF with $\mathcal{Y} = \{0,1\}^{10}$.

Is the derived MAC $\mathcal{I}_F$ a secure MAC system?

A bad example

Suppose $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ is a secure PRF with $\mathcal{Y} = \{0,1\}^{10}$.

Is the derived MAC $\mathcal{I}_F$ a secure MAC system?

No tags are too short: anyone can guess the tag for any msg

$$MACadv[\mathcal{A}, \mathcal{I}_F] = 1/1024.$$

Security

Theorem

If F defined over $(\mathcal{K}, \mathcal{X}, \mathcal{Y})$ is a secure PRF and $|\mathcal{Y}|$ is super-poly, then $\mathcal{I}_F$ is a secure MAC.
In particular, for every eff. MAC adversary $\mathcal{A}$ attacking $\mathcal{I}_F$ there exists an eff. PRF adversary $\mathcal{B}$ attacking F s.t.:

$$MACadv[\mathcal{A}, \mathcal{I}_F] \leq PRFadv[\mathcal{B}, F] + 1/|\mathcal{Y}|$$

$\mathcal{I}_F$ is secure as long as $|\mathcal{Y}|$ is large, say $|\mathcal{Y}| = 2^{80}$.

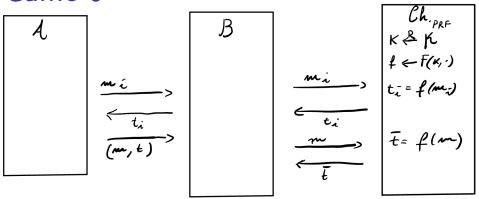
Corollary (Truncating MACs)

If $\mathcal{I} = (S, V)$ is a MAC based on a secure PRF outputting n-bit tags, then the truncated MAC outputting $w < n$ bits is secure.

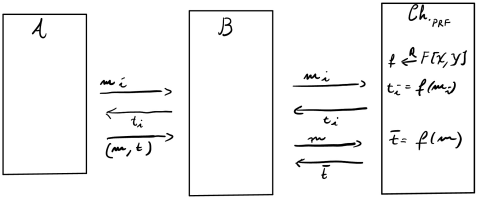
Proof

Idea: $\mathcal{B}$ uses $\mathcal{A}$ to understand if he is interacting with a challenger implementing a random function or F.

Game 0



Game 1



If $t = \bar{t}$ then $\mathcal{B}$ output $\hat{b} = 0$, otherwise 1.
Let $W_b = \{\mathcal{A}$ outputs $t = f(m)$ in Game $b\}$, $(b \in \{0, 1\})$.
Note: $Pr[W_0] = MACadv[\mathcal{A}, \mathcal{I}_F]$ and $Pr[W_1] = 1/|\mathcal{Y}|$.

$$\Downarrow$$

$$MACadv[\mathcal{A}, \mathcal{I}_F] = Pr[W_0] \leq |Pr[W_0] - Pr[W_1]| + Pr[W_1] = PRFadv[\mathcal{B}, F] + 1/|\mathcal{Y}|.$$

Example

If we use AES as PRF then we can create MAC for messages 128-bits long

$$\text{length message} = \text{length output of } F$$

Question: how to create secure MAC for long messages?

Goal: given a secure PRF on short inputs construct a secure PRF on long inputs.

Prefix-free secure PRFs

Prefix-free secure PRFs

Definition

Let $x = (a_1, ..., a_r)$ and $y = (b_1, ..., b_s)$. x is called a proper prefix of y if $r \leq s$ and $b_j = a_j$ for all $j = 1, ..., r$

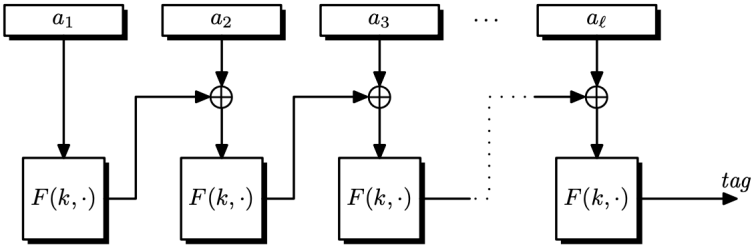
Let F be PRF defined over $(\mathcal{K}, \mathcal{X}^{\leq \ell}, \mathcal{Y})$. F is said **prefix-free secure** PRF is indistinguishable from a random function when considering prefix-free adversary. That is, the queries of the adversary are such that no one is a proper prefix of another.

Raw CBC

Let F be a PRF over $(\mathcal{K}, \mathcal{X}, \mathcal{X})$, with $\mathcal{X} = \{0,1\}^n$. We define F_{CBC} over $(\mathcal{K}, \mathcal{X}^{\leq \ell}, \mathcal{X})$

- Input: k, $m = (a_1, ..., a_v)$
- $t = 0$
- for $i = 1, ..., v$ do:
  $t = F(k, a_i \oplus t)$
- output t

Raw CBC



Raw CBC vs CBC enc

F_{CBC} is similar to CBC mode encryption, but with two important differences:

- F_{CBC} uses a fixed IV, i.e. IV=0, CBC mode enc. uses a random IV;
- F_{CBC} does not output any intermediate values along the CBC chain.

Theorem

If F is a secure PRF over $(\mathcal{K}, \mathcal{X}, \mathcal{X})$, $\mathcal{X} = \{0,1\}^n$ and 2^n is super-poly, thenF_{CBC} is a prefix-free secure PRF.
In particular, for every eff. prefix-free PRF adversary $\mathcal{A}$ attacking F_{CBC} there exists an eff. PRF adversary $\mathcal{B}$ attacking F s.t.:

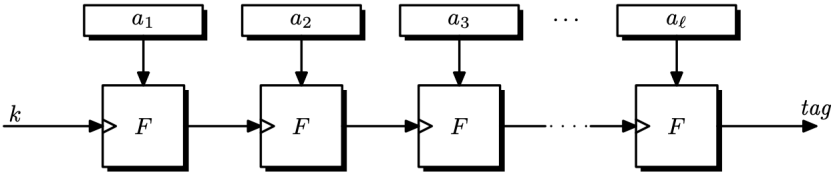
$$PRF^{pf}adv[\mathcal{A}, F_{CBC}] \leq PRFadv[\mathcal{B}, F] + \frac{(\ell \cdot Q)^2}{2|\mathcal{X}|}.$$

Cascade prefix-free secure PRF

Let F be a PRF over $(\mathcal{K}, \mathcal{X}, \mathcal{K})$. We define F^*, the **cascade of** F, over $(\mathcal{K}, \mathcal{X}^{\leq \ell}, \mathcal{K})$

- Input: k, $m = (a_1, ..., a_v)$
- $t = k$
- for $i = 1, ..., v$ do:
  $t = F(t, a_i)$
- output t

Cascade



Theorem

If F is a secure PRF over $(\mathcal{K}, \mathcal{X}, \mathcal{K})$, then F^* is a prefix-free secure PRF. In particular, for every eff. prefix-free PRF adversary $\mathcal{A}$ attacking F_{CBC} there exists an eff. PRF adversary $\mathcal{B}$ attacking F s.t.:

$$PRF^{pf}adv[\mathcal{A}, F^*] \leq Q \cdot \ell PRFadv[\mathcal{B}, F].$$

Raw CBC vs. Cascade

Remark

- *CBC uses a fixed key k at any application of F.*
- *Cascade uses a different key at each round.*
- *constant re-keying in cascade may result in worse performance than CBC*
- *no additive error term in security bound for cascade $\Rightarrow$ cascade construction secure even if F (secure) PRF with a small domain $\mathcal{X}$*

Extension attacks

MACs derived from CBC and cascade are insecure, that is, CBC and cascade are **not** secure PRFs

An attack on raw CBC

The forger for raw CBC MAC works as follows:

- select any $a_1 \in \mathcal{X}$ and send to the challenger
- obtain t tag for a_1
- let $a_2 = a_1 \oplus t$ and $m = (a_1, a_2)$.
- output (m, t) as forgery

Remark

t is a MAC for m. Indeed, $t = F(k, a_1)$ and $a_1 = a_2 \oplus F(k, a_1)$. So,

$$F_{CBC}(k, m) = F(k, F(k, a_1) \oplus a_2) = F(k, a_1) = t.$$

Extension attack on cascade

Given $F^*(k, m)$ for some message $m \in \mathcal{X}^{\leq \ell}$, anyone can compute

$$t' = F^*(k, m||m')$$

for any m', without knowledge of k. $F^*(k, m||m') = F^*(t, m')$.

This extension property immediately implies that the MAC derived from F^* is terribly insecure.

ECBC, NMAC, CMAC and PMAC

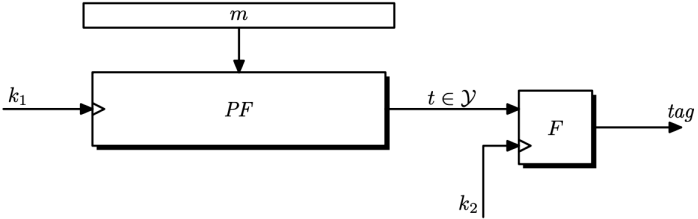
From prefix-free secure PRF to secure PRF

- Encrypted PRF: encrypt the short output of PF with another PRF.
- Prefix-free encoding: encode the input to PF so that no input is a prefix of another.
- CMAC: a more efficient prefix-free encoding using randomization.

Encrypted PRF

Let PF be a PRF over $(\mathcal{K}, \mathcal{X}^{\leq \ell}, \mathcal{Y})$ and let F be a PRF mapping $\mathcal{Y}$ to $\mathcal{T}$. Define

$$EF((k_1, k_2), m) = F(k_2, PF(k_1, m))$$



Extendable PRF

F defined over $(\mathcal{K}, \mathcal{X}^{\leq \ell}, \mathcal{Y})$. F is an *extendable* PRF if for all $k \in \mathcal{K}$, $x, y \in \mathcal{X}^{\leq \ell - 1}$, and $a \in \mathcal{X}$ we have:

$$\text{if } F(k, x) = F(k, y) \text{ then } \mathrm{F(k,\ x \mid\mid a) = F(k,\ y \mid\mid a)}.$$

Example

Raw CBC and cascade are extendable PRF

Theorem

Let PF be an extendable and prefix-free secure PRF over $(\mathcal{K}_1, \mathcal{X}^{\leq \ell+1}, \mathcal{Y})$*, where* $|\mathcal{Y}|$ *is super-poly. Let F be a secure PRF defined over* $(\mathcal{K}_2, \mathcal{Y}, \mathcal{T})$*. Then EF is a secure PRF defined over* $(\mathcal{K}_1 \times \mathcal{K}_2, \mathcal{X}^{\leq \ell}, \mathcal{T})$*.*
In particular,

$$PRFadv[\mathcal{A}, EF] \leq PRFadv[\mathcal{B}_1, F] + PRF^{pf}adv[\mathcal{B}_2, PF] + \frac{Q^2}{2|\mathcal{Y}|}.$$

The bound is tight

Assume that:

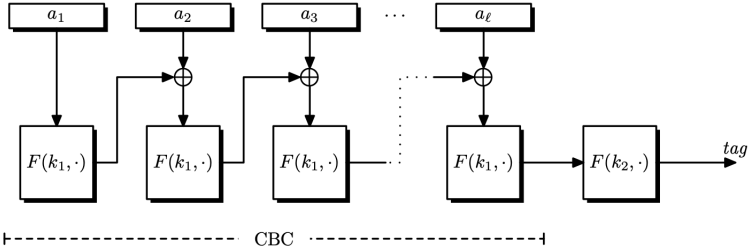
- $\mathcal{Y} = \mathcal{T}$,
- F is a block cipher (PRP),
- $|\mathcal{X}|$ is not too small

Then:

- Adv. picks Q random inputs $x_1, ..., x_Q \in \mathcal{X}$
- By the birthday paradox, if $Q \approx \sqrt{|\mathcal{Y}|} \Rightarrow \exists x_i \neq x_j$ and $PF(k_1, x_i) = PF(k_1, x_j)$
- PF extendable $\Rightarrow$ for any a, $PF(k_1, x_i||a) = PF(k_1, x_j||a)$
- Since F is a permutation $t_i = t_j$ and we can detect the collision.
- Adv. asks a tag for $x_i||a$ obtaining a valid tag for $x_j||a$.

Encrypted-CBC (ECBC)

ECBC is raw CBC and the final encryption. Consequently, ECBC is defined over $(\mathcal{K}^2, \mathcal{X}^{\leq \ell}, \mathcal{X})$.



Security of ECBC

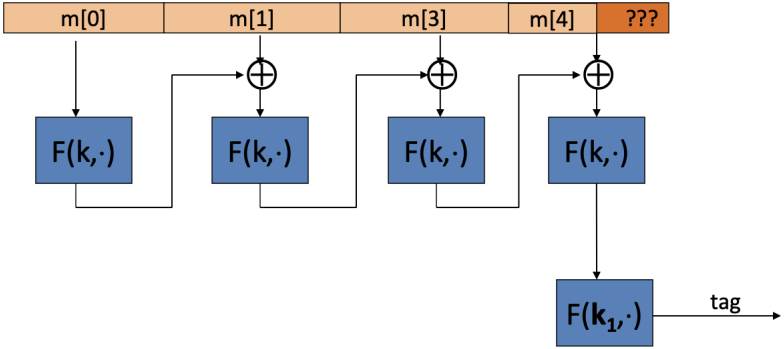
Theorem

For every eff. Q-query PRF adv. $\mathcal{A}$ attacking ECBC there exists an eff. adversary $\mathcal{B}$ s.t.:

$$PRFadv[\mathcal{A}, ECBC] \leq 2 \cdot PRFadv[\mathcal{B}, F] + \frac{Q^2(\ell+1)^2 + Q^2}{2|\mathcal{X}|}.$$

ECBC Padding

What if message length is not multiple of block-size?



ECBC Padding

Bad idea: pad *m* with 0's

Q: Is the resulting MAC secure?

ECBC Padding

Bad idea: pad m with 0's

Q: Is the resulting MAC secure?

A: No, given tag on msg m attacker obtains tag on $m||0$,

$$pad(m) = pad(m||0)$$

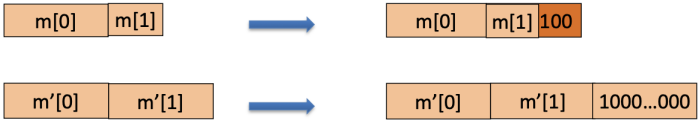
ECBC Padding

For security, padding must be invertible!

$$m_0 \neq m_1 \Rightarrow pad(m_0) \neq pad(m_1)$$

ISO: pad with "100000". Add new dummy block if needed.

- The "1" indicates beginning of pad.

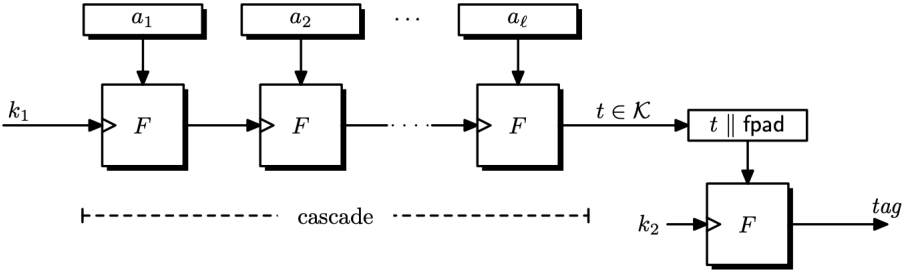


Nested MAC (NMAC)

Applying EF construction to cascade we obtain NMAC.

- We wish to use the same underlying PRF F for the cascade construction and for the final encryption.
- we need to embed the output of cascade into $\mathcal{X}$, i.e. $|\mathcal{K}| \leq |\mathcal{X}|$.
- Suppose $\mathcal{K} = \{0,1\}^{\kappa}$, $\mathcal{X} = \{0,1\}^{n}$, then $t \mapsto t||fpad$, where fpad is a fixed pad of length $n - \kappa$

NMAC


a_1
a_2
$\cdots$
a_ℓ
k_1
F
F
F
$t \in \mathcal{K}$
$t \parallel$ fpad
cascade
k_2
F
tag

NMAC security

Theorem

For every eff. Q-query PRF adv. $\mathcal{A}$ attacking NMAC there exists an eff. adversary $\mathcal{B}$ s.t.:

$$PRFadv[\mathcal{A}, NMAC] \leq Q(\ell + 1) \cdot PRFadv[\mathcal{B}, F] + \frac{Q^2}{2|\mathcal{K}|}.$$

Prefix-free encodings

- $S \subseteq \mathcal{X}^{\leq \ell}$ is a prefix-free set if no element in S is a proper prefix of any other.

  Example: if $(x_1, x_2, x_3) \in S$. Then neither x_1 nor (x_1, x_2) are in S.

- We say that a function $pf : \mathcal{M} \to \mathcal{X}^{\leq \ell}$ is a prefix-free encoding if pf is injective and the image of pf is a prefix-free set.

Theorem

If PF is a prefix-free secure PRF and pf is a prefix-free encoding then $F(k, m) := PF(k, pf(m))$ is a secure PRF.

Prefix free encodings

prepend length: $\mathcal{X} = \{0,1\}^n$, $\mathcal{M} = \mathcal{X}^{\leq \ell - 1}$, $m = (a_1, ..., a_v) \in \mathcal{M}$. Then

$$pf(m) = (\langle v \rangle_2, a_1, ..., a_v),$$

where $\langle v \rangle_2$ is the binary representation of v.

Note: appending the length is not secure!

stop bits: $\bar{\mathcal{X}} = \{0,1\}^{n-1}$, $\mathcal{M} = \bar{\mathcal{X}}^{\leq \ell}$, $m = (a_1, ..., a_v) \in \mathcal{M}$. Then

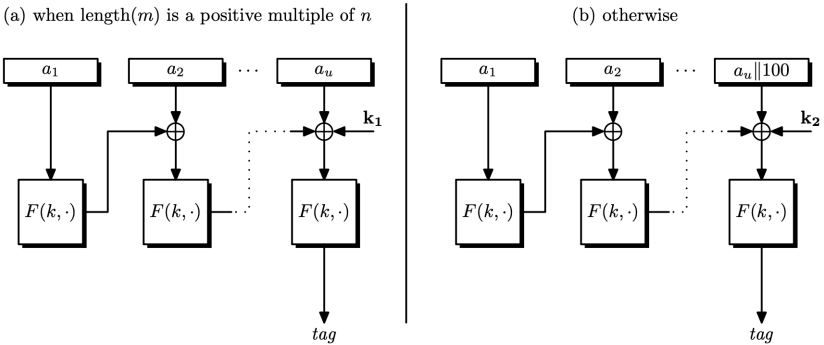
$$pf(m) = ((a_1||0), (a_2||0)..., (a_v||1)),$$

CMAC (NIST standard)

Cipher-based MAC (CMAC) is a variant of ECBC adopted by NIST in 2005

- key $= (k, k_1, k_2)$
- No final encryption step
- No dummy block

CMAC



- extension attack thwarted by last keyed XOR
- ambiguity on the padding resolved by use of k_1 or k_2

parallel MAC

ECBC, CMAC, and NMAC, are sequential:

Block number i cannot be processed before block number $i-1$ is finished

Q: Can we build a parallel MAC from a small PRF?

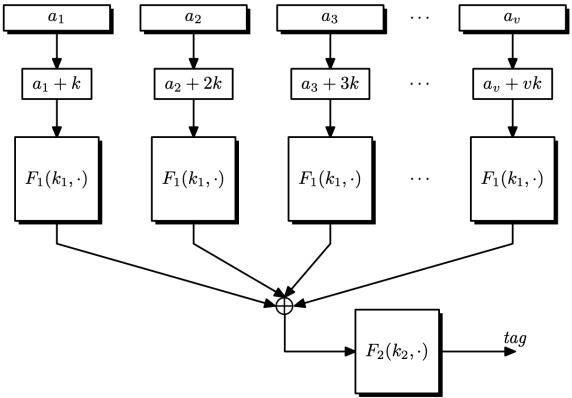
PMAC$_p$

F_1 PRF over $(\mathcal{K}_1, \mathbb{Z}_p, \mathcal{Y})$, where p is a prime, and $\mathcal{Y} = \{0,1\}^n$. F_2 PRF over $(\mathcal{K}_2, \mathcal{Y}, \mathcal{Z})$. PMAC$_p$ defined over $(\mathbb{Z}_p \times \mathcal{K}_1 \times \mathcal{K}_2, \mathbb{Z}_p^{\leq \ell}, \mathcal{Y})$ works as follows:

- input: $m = (a_1, ..., a_v) \in \mathbb{Z}_p^v$, and $key = (k, k_1, k_2) \in \mathbb{Z}_p \times \mathcal{K}_1 \times \mathcal{K}_2$
- $t \leftarrow 0_n$, $mask \leftarrow 0 \in \mathbb{Z}_p$
- for $i = 1, ..., v$ do

  $mask = i \cdot k$

  $t \leftarrow t \oplus F_1(k_1, mask + a_i)$
- output $F_2(k_2, t)$


a_1
a_2
a_3
$\cdots$
a_v
$a_1 + k$
$a_2 + 2k$
$a_3 + 3k$
$\cdots$
$a_v + vk$
$F_1(k_1, \cdot)$
$F_1(k_1, \cdot)$
$F_1(k_1, \cdot)$
$\cdots$
$F_1(k_1, \cdot)$
$F_2(k_2, \cdot)$
tag

Remark

- *we can use $\mathbb{Z}_{2^n}$ instead of $\mathbb{Z}_p$. In this case we need k odd.*
- *XOR version of PMACp, PMAC (Rogaway 2002):*
  - *$\mathbb{Z}_p$ replaced with $GF(2^n)$ and the mask is $\gamma_i \cdot k$, where $\gamma_i \in GF(2^n)$ fixed constants.*
  - *elements in $GF(2^n)$ can be represented as n-bit strings, and sum in $GF(2^n)$ is the XOR*
  - *$F_1 = F_2 = F$*
  - *$k = F(k_1, 0)$ and $k_2 = k_1$ (shorter secret key)*
  - *similar trick as CMAC when padding to avoid dummy block*

PMAC security

Theorem

If F_1 and F_2 are secure PRFs, and $|\mathcal{Y}|$ and the prime p are super-poly, then $PMAC_p$ is a secure PRF. In particular

$$PRFadv[\mathcal{A}, PMAC_p] \leq PRFadv[\mathcal{B}_1, F_1] + PRFadv[\mathcal{B}_2, F_2] + \frac{Q^2}{2|\mathcal{Y}|} + \frac{Q^2\ell^2}{2p}.$$

PMAC is incremental

- Suppose Bob computes the tag t for some long message $m = (a_1, a_2, ..., a_v)$
- later he changes one block in m and wants to recompute the tag of this new message $m' = (a_1, ..., a'_i, ..., a_v)$.
- using CBC-MAC the tag t is of no help: we need to re-compute the tag
- Whit PMAC_p we can do much better:
  - Suppose F_2 PRP (i.e. exists F_2^{-1})

$$PMAC_p(k, k_1, k_2, m') = F_2(k_2, F_2^{-1}(k_2, t) \oplus F_1(k_1, a_i + i \cdot k) \oplus F_1(k_1, a'_i + i \cdot k))$$

Hash-MAC (HMAC)

Most widely used MAC on the Internet...

...but, we first we need to discuss hash function.